CryptoVault

Mobile Application Security Audit & Remediation Report

OWASP Mobile Top 10 · 10 vulnerabilities / 16 sub-vulnerabilities

MobSF Security Score 24/100 → 61/100

Instructor: Mme Manel Elloumi

Package: tn.iit.cryptovault · Platform: Flutter / Android

Team: _____ · Date: June 2026

Capture #1 — to insert

CryptoVault login screen (cover/intro shot).

Figure 1. CryptoVault login screen (cover/intro shot). *Why it proves the flaw: Establishes the application under audit.*

Table of contents

Table of contents 2
Executive summary 3
Audit methodology 4
Detailed findings 5
- V1 — Hardcoded secrets in the binary 5
- V2 — Insecure local storage 5
- V3 — TLS validation disabled 6
- V4 — Sensitive data written to logs 7
- V5a — Client-side JWT role escalation 8
- V5b — Logout does not clear the session 9
- V6 — IDOR via sequential identifiers 10
- V7 — WebView XSS with a native bridge 12
- V8 — Insecure Android manifest configuration 13
- V9a — MD5 PIN hashing 14
- V9b — AES-ECB with a hardcoded key 14
- V9c — Predictable random number generation 16
- V10a — Hidden debug panel 17
- V10b — Hardcoded backdoor account 18
- V10c — Unauthenticated admin/test endpoints 18

MobSF before / after comparison 20
Limits of automated analysis 23
Attack chains 24
Summary & remediation roadmap 25
Annexes 26

Executive summary

CryptoVault is a Flutter-based cryptocurrency wallet for Android. This audit examined the application across the full OWASP Mobile Top 10 and identified **10 main vulnerabilities comprising 16 sub-vulnerabilities**, ranging from hardcoded secrets and disabled TLS validation to a hidden debug panel, a hardcoded backdoor account, and unauthenticated administrative endpoints. Several findings chain together to produce complete compromise of both the device-side wallet and the server-side data.

Every finding was reproduced with concrete proof (binary extraction, network interception, on-device storage dumps, and scripted cryptographic attacks) and then **remediated** in a hardened build. The remediation included a full rewrite of the backend to a real authenticated service (Express + RS256 JWT over HTTPS with certificate pinning), migration of local storage to the Android Keystore, modern authenticated cryptography, and removal of all backdoors and debug surfaces.

A static scan with MobSF v4.5.0 confirms the improvement: the security score rose from **24/100** on the vulnerable build to **61/100** on the hardened build (+154%). Every HIGH-severity finding attributable to application code was eliminated; the single residual HIGH is a documented false-positive inside a third-party library (see the limits section).

> **Language note.** This report is written in English. The original assignment brief (CDC) was specified in French; a French translation pass can be added if required, with figures unchanged.

Audit methodology

The audit followed a five-stage pipeline, each stage targeting a distinct layer of the application and producing reproducible evidence:

Stage	Focus	Primary tools	Evidence
1 · Static / binary	Secrets, obfuscation, manifest	apktool, strings	Figures 2, 3
2 · Storage	At-rest data, backups	adb run-as, adb backup	Figures 4, 5, 6
3 · Network	TLS, interception, IDOR	Burp Suite	Figures 7, 8, 9
4 · Dynamic	Runtime, WebView, deep links	adb logcat, am, device	Figures 10–14, 18–22
5 · Cryptography	Hashing, encryption, RNG	Python, pycryptodome	Figures 15, 16, 17

A complementary automated scan (MobSF v4.5.0, Docker) was run on both the vulnerable and the hardened APKs to cross-confirm findings and quantify the improvement.

Detailed findings

Each finding follows the four-part structure: technical description, proof of concept, concrete impact, and the remediation actually applied — with the verbatim secured code taken from the hardened build.

V1 — Hardcoded secrets in the binary

CRITICAL · OWASP M8 / M9

Location: `lib/core/config.dart → libapp.so`

1 · Technical description

The vulnerable build compiles six secrets directly into the application: a JWT signing secret, third-party API keys (Coinbase, Etherscan), the AES key used for seed-phrase encryption, an internal admin URL, and a developer master token. Because Flutter compiles Dart to the native shared object `libapp.so`, these values travel inside every shipped APK and can be recovered with nothing more than the `strings` utility.

2 · Proof of concept

Running `strings libapp.so | grep -Ei 'jwt|key|token|coinbase|etherscan'` on the extracted APK prints all six secrets in cleartext, including `cryptovault_jwt_s3cr3t` and the AES seed key.

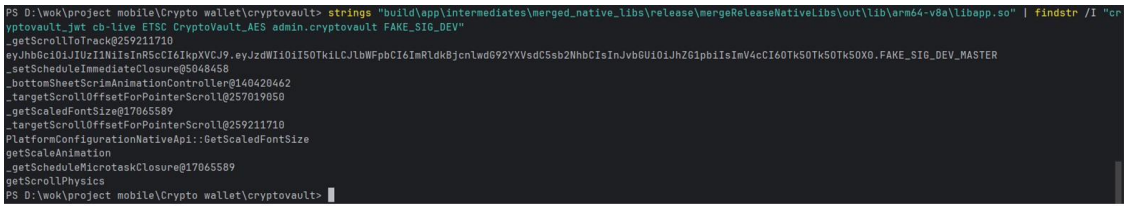


Figure 2. strings extraction of libapp.so revealing all six embedded secrets. *Why it proves the flaw: The secrets are readable directly from the shipped binary, with no reverse-engineering effort.*

3 · Concrete impact

An attacker who downloads the APK recovers the JWT secret (enabling token forgery — see V5a), the AES key (enabling decryption of any stored seed phrase — see V9b), and the developer master token (enabling the V10b backdoor). A single passive extraction compromises authentication and at-rest encryption simultaneously.

4 · Remediation (applied)

Secrets are removed from source entirely. The host is injected at build time via `--dart-define`; any real third-party key is fetched through an authenticated server call, never compiled in. R8 minification/obfuscation is enabled for release builds. Post-fix, `strings app-release.apk | findstr cryptovault_jwt` returns nothing.

Vulnerable code (before)

Secured code (after)

V2 — Insecure local storage

CRITICAL · OWASP M2

Location: `lib/core/storage.dart`

1 · Technical description

The session is persisted to Flutter's `SharedPreferences`, an unencrypted XML file in the app sandbox. Seven sensitive fields — JWT, user role, balance, wallet address, seed phrase, and KYC id — are written in plaintext.

2 · Proof of concept

With `adb shell run-as tn.iit.cryptovault cat .../shared_prefs/FlutterSharedPreferences.xml` the full session is dumped while logged in; the seed phrase and token are plainly visible.

```
PS D:\wok\project mobile\Crypto wallet\cryptovault> adb shell run-as tn.iit.cryptovault cat /data/data/tn.iit.cryptovault/shared_prefs/FlutterSharedPreferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="flutter.user_role">user</string>
    <string name="flutter.user_email">ali@cryptovault.tn</string>
    <string name="flutter.seed_phrase">legal winner thank year wave sausage worth useful legal winner thank yellow</string>
    <string name="flutter.auth_token">eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxI1wiZW1haWwiOiJhbGlAY3JScHRvdmF1bHQudG4iLCJyb2xlIjoidXNlciIsImV4cCI6OTk5OTk5OTk5OX0.FAKE_SIG_
ALI</string>
    <string name="flutter.kyc_id">TN-CIN-08123456</string>
    <boolean name="flutter.is_logged_in" value="true" />
    <string name="flutter.balance">VGhpcyBpcyB0aGUgcHJlZml4IGZvciBEb3VibGUu12450.75</string>
    <string name="flutter.wallet_address">bc1qxy2kgdygjrsqtzq2n0yrf2493p83kkfjhx0wlh</string>
    <long name="flutter.user_id" value="1" />
</map>
```

Figure 5. run-as dump of FlutterSharedPreferences.xml showing seven plaintext fields. *Why it proves the flaw: Sensitive wallet data sits unencrypted in the app sandbox, readable on a debuggable or rooted device.*

3 · Concrete impact

Any process with sandbox access (a debuggable build, a rooted device, or a backup — see V8) reads the seed phrase and drains the wallet. Storing the role here also enables the V5a privilege escalation.

4 · Remediation (applied)

Storage is migrated to `flutter_secure_storage` with `encryptedSharedPreferences: true`, backing all writes with the Android Keystore (AES-256-GCM via androidx EncryptedSharedPreferences). Only the access token, refresh token and user id are persisted; balance, address and KYC are re-fetched from the API; the seed phrase never leaves the device in cleartext.

Vulnerable code (before)

Secured code (after)

V3 — TLS validation disabled

CRITICAL · OWASP M3

Location: `lib/core/network.dart, lib/main.dart`

1 · Technical description

The Dio HTTP client installs a `badCertificateCallback` that returns `true` for every certificate, accepting any TLS peer. A dormant `BurpProxyOverride` in `main.dart` additionally routes traffic through an attacker proxy.

2 · Proof of concept

Pointing the device at a Burp Suite proxy with its self-signed CA, the entire login flow is intercepted in cleartext — the `Authorization: Bearer …` header is fully readable in the HTTP history.

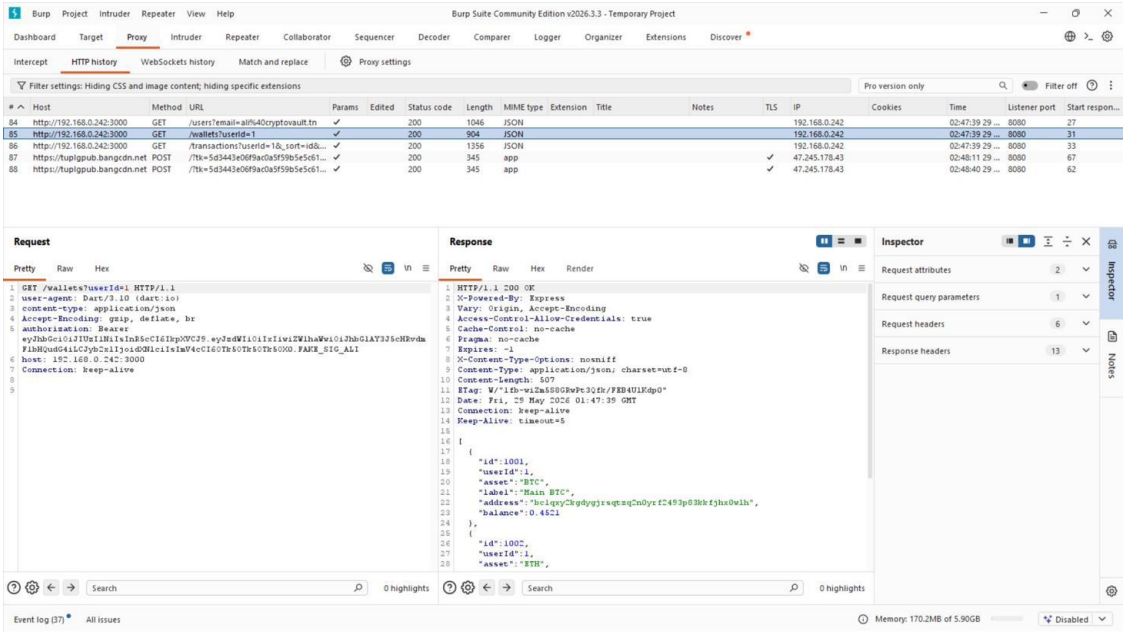


Figure 7. Burp HTTP history showing the intercepted login and bearer token. *Why it proves the flaw: Disabled certificate validation lets a man-in-the-middle read and modify all traffic.*

3 · Concrete impact

A network attacker (rogue Wi-Fi, malicious proxy) reads credentials and tokens and tampers with responses. Combined with V4 (token also logged) and V8 (cleartext permitted), exfiltration is trivial.

4 · Remediation (applied)

The bad-certificate callback is removed and `BurpProxyOverride` deleted. The client now runs over HTTPS and pins the server certificate: a `SecurityContext(withTrustedRoots:false)` trusts only the bundled `api.cryptovault.crt`, so any other certificate (including Burp's) fails the handshake.

Vulnerable code (before)

Secured code (after)

V4 — Sensitive data written to logs

HIGH · OWASP M1

Location: `26 print() sites across lib/`

1 · Technical description

The app logs credentials, full JWTs, the seed phrase, a wallet private key, and transaction nonces through 26 `print('[CV-…]')` statements that run in every build.

2 · Proof of concept

`adb logcat` during login and a send operation captures `[CV-AUTH] Password: Ali2026!`, the complete bearer token, the seed phrase, and `[CV-SEND] PrivateKey: L1aW…`.

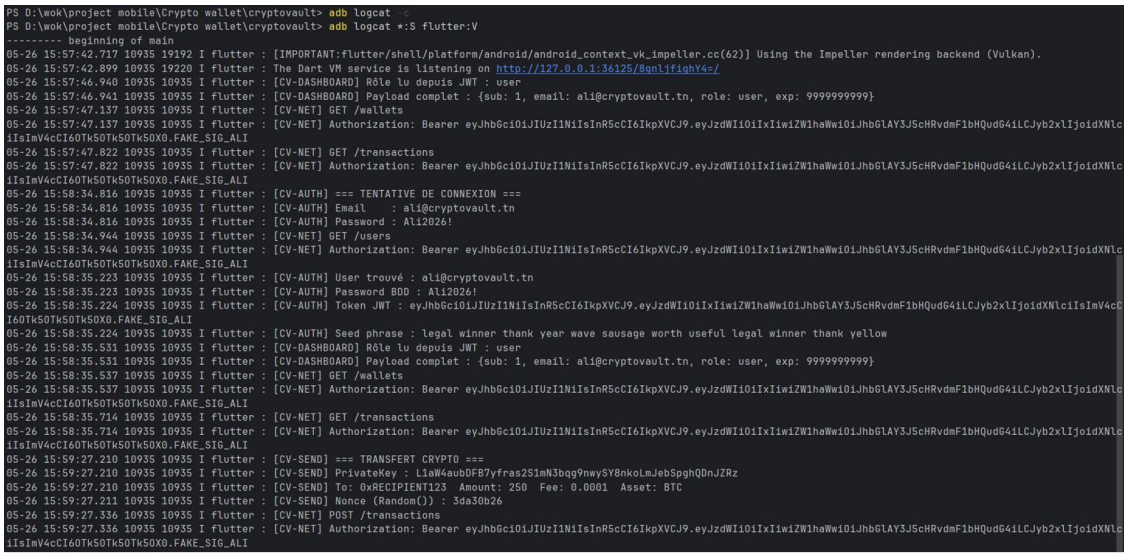


Figure 10. logcat capturing password, bearer token, seed phrase and private key. *Why it proves the flaw: Secrets are emitted to a system-wide log readable by other apps holding READ_LOGS or via adb.*

3 · Concrete impact

Any app with log access, or anyone with momentary USB access, harvests live credentials and keys. The nonce leaked here also feeds the V9c predictability proof.

4 · Remediation (applied)

All sensitive prints are removed. Network logging is reduced to metadata only (method + path), gated behind `kDebugMode` so nothing is emitted in release, and ProGuard strips `android.util.Log` calls at release build time.

Vulnerable code (before)

Secured code (after)

V5a — Client-side JWT role escalation

HIGH · OWASP M4

Location: `lib/features/dashboard/dashboard_page.dart`

1 · Technical description

Authorization is decided on the client: the dashboard decodes the JWT locally with `JwtDecoder` and trusts the `role` claim without any signature verification. Since the token is self-issued (fake signature) and stored in cleartext (V2), the role is fully attacker-controlled.

2 · Proof of concept

Editing the stored token's payload to `"role":"admin"` (re-written with a Python one-liner that preserves the fake signature) makes the admin panel appear on Ali's account.

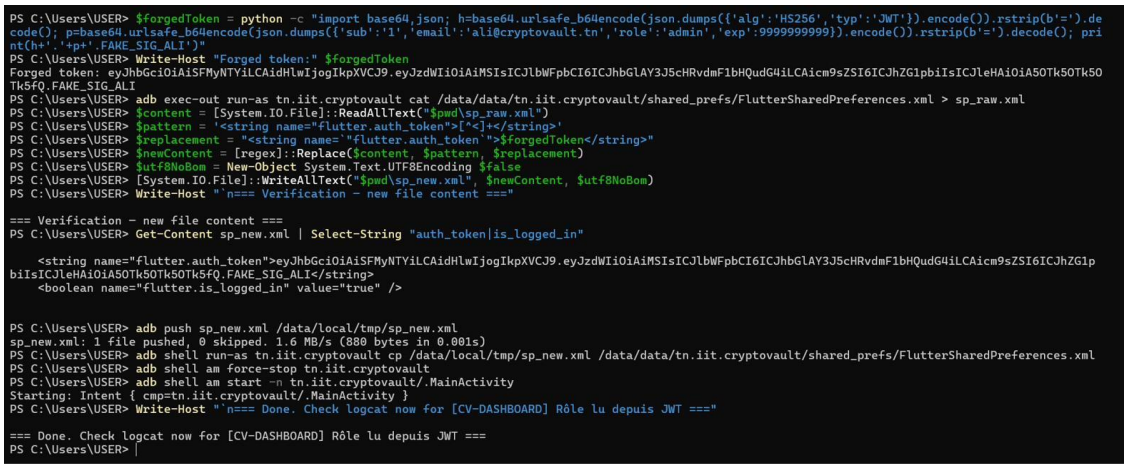


Figure 11. Forged token rewritten with role:admin and the resulting prefs file. *Why it proves the flaw: The role claim is trusted with no server-side check, so a self-edited token grants admin.*

Capture #12 — to insert

Dashboard showing the red Administrator panel after the forged token.

Figure 12. Dashboard showing the red Administrator panel after the forged token. *Why it proves the flaw: The escalated role unlocks admin-only UI and actions end-to-end.*

3 · Concrete impact

A standard user elevates to admin and reaches the unauthenticated admin endpoints (V10c). With V5b, the escalation also survives logout.

4 · Remediation (applied)

The client no longer decodes or trusts the role. Authorization is server-side: the backend verifies the RS256 signature on every request, and the UI asks `GET /admin/whoami` (authoritative) before showing admin features. A self-edited token fails signature verification and is rejected.

Secured code (after)

V5b — Logout does not clear the session

HIGH · OWASP M4

Location: `lib/core/storage.dart`

1 · Technical description

Logout flips an `is_logged_in` boolean but leaves the token, seed phrase and KYC id in storage, and never revokes the token server-side.

2 · Proof of concept

Dumping the prefs file after logout shows every sensitive field still present; only `is_logged_in` changed to false.

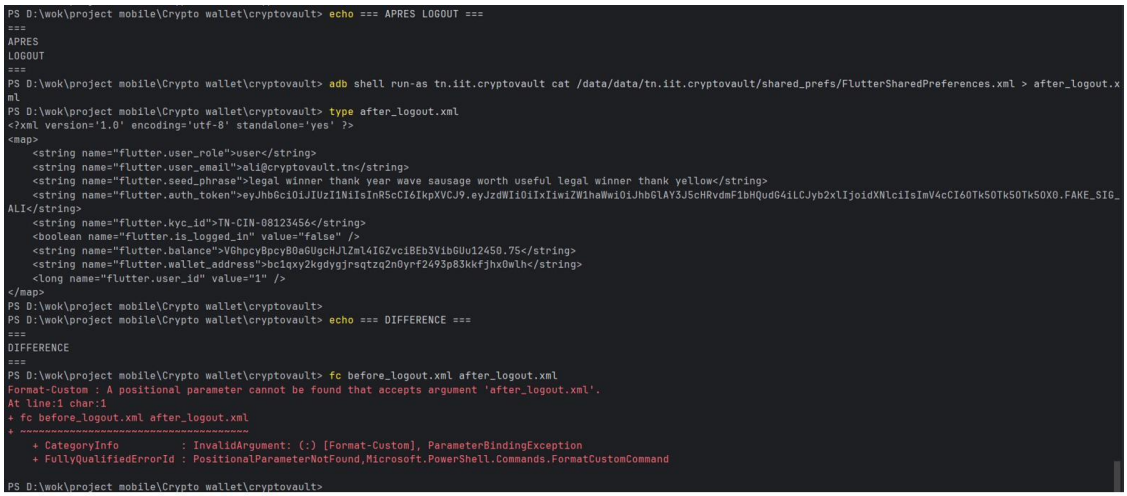


Figure 6. prefs file after logout — token, seed and KYC still present. *Why it proves the flaw: Logout is cosmetic; the credentials needed to impersonate the user remain on disk and valid.*

3 · Concrete impact

A shared or stolen device still holds a usable token after the user 'logs out'; combined with V5a, a forged admin token persists indefinitely.

4 · Remediation (applied)

Logout now calls `POST /auth/revoke` to blacklist the refresh token server-side, then wipes the Keystore with `deleteAll()`. No usable credential remains locally or server-side.

Vulnerable code (before)

Secured code (after)

V6 — IDOR via sequential identifiers

HIGH · OWASP M6

Location: `server, lib/features/transactions/transactions_page.dart`

1 · Technical description

Data endpoints trust a client-supplied `userId` query parameter and identifiers are sequential integers. A user can request any other user's data simply by changing the number.

2 · Proof of concept

Replaying Ali's authenticated request against `/transactions?userId=2` returns Sarra's transactions; `userId=3` reaches the admin. Burp Intruder enumerates ids 1–10 and the response length column reveals which accounts exist.

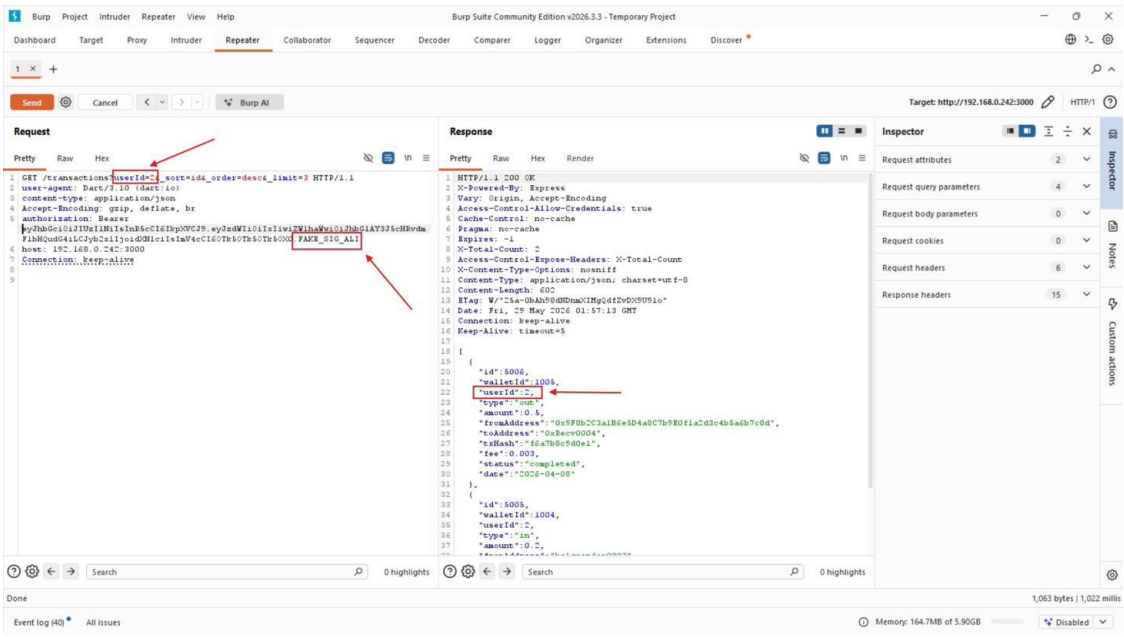


Figure 8. Ali's token replayed against userId=2 returning Sarra's data. *Why it proves the flaw: The server honours an attacker-chosen userId, exposing other users' records.*

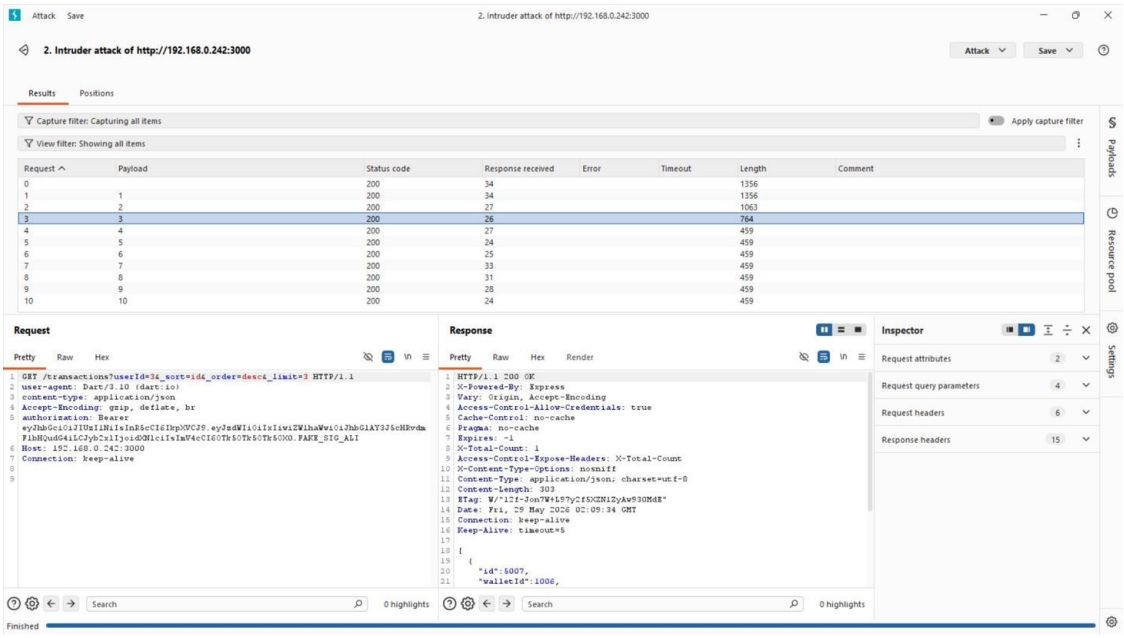


Figure 9. Burp Intruder enumerating userId 1–10 by response length. *Why it proves the flaw: Sequential ids plus no ownership check enable full account enumeration.*

3 · Concrete impact

Complete horizontal and vertical data exposure — every user's balances, transactions and wallet are readable by any authenticated user.

4 · Remediation (applied)

The rewritten Express backend derives the effective user from the verified JWT `sub` claim and ignores any client-supplied `userId` (a normal user can only ever read their own records; only an admin may scope by id). The client no longer sends a user id at all.

Vulnerable code (before)

Secured code (after)

V7 — WebView XSS with a native bridge

HIGH · OWASP M7

Location: `lib/features/support/support_page.dart`

1 · Technical description

The support screen renders user input into a WebView with JavaScript enabled and exposes a native JavaScript channel (`CryptoVault`). Unescaped input executes as script, and that script can call back into native code.

2 · Proof of concept

Submitting a `<script>` payload and an `<img onerror>` payload both fire JS alerts; a payload calling `CryptoVault.postMessage('PWNED')` crosses the bridge and triggers native code (a SnackBar + logcat entry).

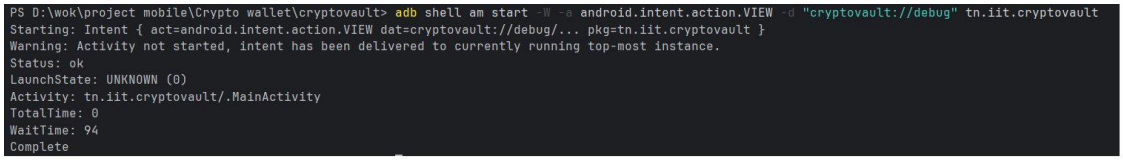


Figure 13. Injected script and img-onerror payloads firing in the WebView. *Why it proves the flaw: Unescaped user input is executed as JavaScript inside the app.*

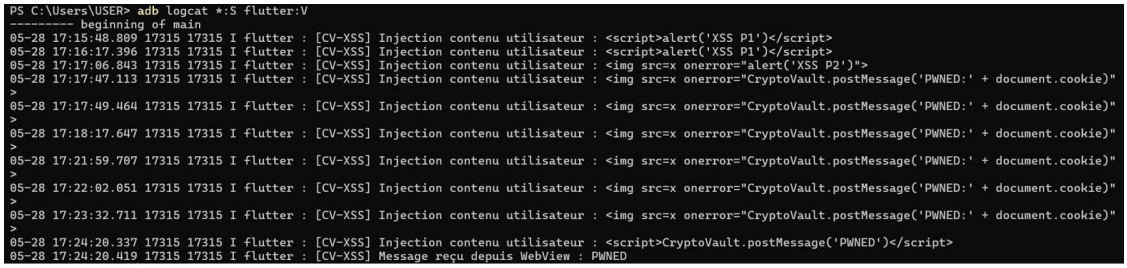


Figure 14. JS-to-native bridge crossed: 'Message natif reçu : PWNED'. *Why it proves the flaw: Injected script reaches native code through the exposed JS channel.*

3 · Concrete impact

Injected script can drive the native bridge to read app state and, with V8's cleartext + V3's MITM, exfiltrate the token to an attacker server.

4 · Remediation (applied)

JavaScript is disabled, the JS channel is removed, user input is HTML-escaped, a restrictive Content-Security-Policy is set, and an input whitelist rejects the characters used in markup injection.

Vulnerable code (before)

Secured code (after)

V8 — Insecure Android manifest configuration

HIGH · OWASP M1 / M8

Location: `AndroidManifest.xml, network_security_config.xml`

1 · Technical description

The manifest requests five permissions a wallet has no need for (READ_SMS, READ_CONTACTS, READ_CALL_LOG, ACCESS_FINE_LOCATION, RECORD_AUDIO) and sets `allowBackup="true"`, `debuggable="true"` and `usesCleartextTraffic="true"`.

2 · Proof of concept

An `adb backup` (permitted by `allowBackup`) is decompressed to reveal the app's private files — including the SharedPreferences XML with the seed phrase (chaining into V2). The manifest dump shows the five dangerous permissions.

```
PS D:\wok\project mobile\Crypto wallet\cryptovault> echo === PERMISSIONS ABUSIVES ===
===
PERMISSIONS
ABUSIVES
===
PS D:\wok\project mobile\Crypto wallet\cryptovault> type app-debug-decoded\AndroidManifest.xml | findstr /I "READ_SMS READ_CONTACTS READ_CALL_LOG ACCESS_FINE_LOCATION RECORD_AUDIO"

    <uses-permission android:name="android.permission.READ_SMS"/>
    <uses-permission android:name="android.permission.READ_CONTACTS"/>
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
    <uses-permission android:name="android.permission.RECORD_AUDIO"/>
    <uses-permission android:name="android.permission.READ_CALL_LOG"/>
PS D:\wok\project mobile\Crypto wallet\cryptovault>
PS D:\wok\project mobile\Crypto wallet\cryptovault> echo === DRAPEAUX DANGEREUX ===
===
DRAPEAUX
DANGEREUX
===
PS D:\wok\project mobile\Crypto wallet\cryptovault> type app-debug-decoded\AndroidManifest.xml | findstr /I "allowBackup debuggable usesCleartextTraffic fullBackupContent"
    <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory" android:debuggable="true" android:extractNativeLibs="false" android
:fullBackupContent="true" android:icon="@mipmap/ic_launcher" android:label="CryptoVault" android:name="android.app.Application" android:networkSecurityConfig="@xml/network_security
_config" android:usesCleartextTraffic="true">
```

Figure 3. Manifest dump: five unjustified dangerous permissions and risky flags. *Why it proves the flaw: The app over-requests sensitive permissions and enables backup/debug/cleartext.*

```
PS D:\wok\project mobile\Crypto wallet\cryptovault> adb backup -f cv_backup.ab -noapk tn.iit.cryptovault
WARNING: adb backup is deprecated and may be removed in a future release
Now unlock your device and confirm the backup operation...
PS D:\wok\project mobile\Crypto wallet\cryptovault> python -c "import zlib;f=open('cv_backup.ab','rb');f.read(24);open('cv.tar','wb').write(zlib.decompress(f.read()))"
PS D:\wok\project mobile\Crypto wallet\cryptovault> tar -xf cv.tar
PS D:\wok\project mobile\Crypto wallet\cryptovault> type apps\tn.iit.cryptovault\sp\FlutterSharedPreferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="flutter.user_role">user</string>
    <string name="flutter.user_email">ali@cryptovault.tn</string>
    <string name="flutter.seed_phrase">legal winner thank year wave sausage worth useful legal winner thank yellow</string>
    <string name="flutter.auth_token">eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwiZW1haWwiOiJhbGlAY3JScHRvdmF1bHQudG4iLCJyb2xlIjoidXNlciIsImV4cCI6OTk5OTk5OTk5OX0.FAKE_SIG_
ALI</string>
    <string name="flutter.kyc_id">TN-CIN-08123456</string>
    <boolean name="flutter.is_logged_in" value="true" />
    <string name="flutter.balance">VGhpcyBpcyB0aGUgcHJlZml4IGZvciBEb3VibGUu12450.75</string>
    <string name="flutter.wallet_address">bc1qxy2kgdygjrsqtzq2n0yrf2493p83kkfjhx0wlh</string>
    <long name="flutter.user_id" value="1" />
</map>
PS D:\wok\project mobile\Crypto wallet\cryptovault>
```

Figure 4. adb backup decompressed, exposing the private prefs with the seed phrase. *Why it proves the flaw: allowBackup makes private app data extractable without root.*

3 · Concrete impact

Over-broad permissions expand the attack surface and harm privacy; `allowBackup` defeats the storage boundary; `debuggable` and cleartext enable the other network and storage attacks.

4 · Remediation (applied)

Permissions are reduced to three normal ones (INTERNET, ACCESS_NETWORK_STATE, USE_BIOMETRIC). `allowBackup=false`, data-extraction rules exclude all app data, the debug flag is left to the build type, cleartext is forbidden in the network security config, and the V10a deep-link filter is removed.

Secured code (after)

V9a — MD5 PIN hashing

MEDIUM · OWASP M5

Location: `lib/core/crypto_service.dart`

1 · Technical description

The PIN is hashed with a single, unsalted MD5 — a fast, broken hash entirely unsuitable for credential storage.

2 · Proof of concept

A Python dictionary attack recovers the PIN `1234` in 1.42 ms at roughly 845,000 hashes/second, matching on the first dictionary entry.

```
PS D:\wok\project mobile\Crypto wallet\cryptovault> python audit_python\v9a_bruteforce_pin.py
[V9a] Target MD5 hash: 81dc9bdb52d04dc20036dbd8313ed055
[V9a] Search space: 10,000 PINs (0000 to 9999)
[V9a] Starting brute force...

[V9a] PIN FOUND   : 1234
[V9a]   Attempts  : 1235
[V9a]   Elapsed   : 1.42 ms (0.0014 s)
[V9a]   Throughput: 870582 hashes/second

[V9a-bonus] Top-10 most common PINs:
   1234 -> 81dc9bdb52d04dc20036dbd8313ed055 <-- MATCH
   1111 -> b59c67bf196a4758191e42f76670ceba
   0000 -> 4a7d1ed414474e4033ac29ccb8653d9b
   1212 -> a01610228fe998f515a72dd730294d87
   7777 -> d79c8788088c2193f0244d8f1f36d2db
   1004 -> fed33392d3a48aa149a87a38b875ba4a
   2000 -> 08f90c1a417155361a5c4b8d297e0d78
   4444 -> dbc4d84bfcfe2284ba11beffb853a8c4
   2222 -> 934b535800b1cba8f96a5d72f72f1611
   6969 -> 7813d1590d28a7dd372ad54b5d29d033

[V9a] Conclusion: MD5 without salt or work factor offers
      ZERO resistance over a 10,000-value space.
```

Figure 15. PIN 1234 cracked in 1.42 ms via MD5 dictionary attack. *Why it proves the flaw: An unsalted fast hash offers essentially no resistance to brute force.*

3 · Concrete impact

A stored PIN hash (recoverable via V2) is cracked instantly, defeating the local PIN protection.

4 · Remediation (applied)

PIN hashing is replaced with Argon2id (memory-hard, salted, with cost parameters), raising the per-guess cost by orders of magnitude and making dictionary attacks impractical.

Vulnerable code (before)

Secured code (after)

V9b — AES-ECB with a hardcoded key

HIGH · OWASP M5

Location: `lib/core/crypto_service.dart`

1 · Technical description

Seed phrases are encrypted with AES in ECB mode using the hardcoded key from V1 and a null IV. ECB leaks structure (identical plaintext blocks produce identical ciphertext) and provides no integrity.

2 · Proof of concept

Encrypting the same seed three times yields three identical ciphertexts (`ct1==ct2==ct3`); using the leaked V1 key, any captured ciphertext is decrypted back to the seed phrase.

```
PS D:\wok\project mobile\Crypto wallet\cryptovault> python audit_python\v9b_aes_ecb.py
[V9b] AES-256 key (extracted via V1) : b'CryptoVault_AES_Seed_Key_2026!!0'
[V9b] Key length : 32 bytes

================================================================
PART 1 — AES-ECB determinism
================================================================

[V9b] Plaintext      : legal winner thank year wave sausage worth useful

[V9b] Encryption #1  : pZ0ogGKzzM87jF88btSF1L8Un5QtPfjWCyFYNJ4qJd/a7MAuu/dn2tH5ZcTRw9o5jcdj+hL/zabGN5+r1kvNyQ==
[V9b] Encryption #2  : pZ0ogGKzzM87jF88btSF1L8Un5QtPfjWCyFYNJ4qJd/a7MAuu/dn2tH5ZcTRw9o5jcdj+hL/zabGN5+r1kvNyQ==
[V9b] Encryption #3  : pZ0ogGKzzM87jF88btSF1L8Un5QtPfjWCyFYNJ4qJd/a7MAuu/dn2tH5ZcTRw9o5jcdj+hL/zabGN5+r1kvNyQ==

[V9b] ct1 == ct2 == ct3 ? True
[V9b] ECB is DETERMINISTIC — an attacker who observes the
      same ciphertext twice knows it represents the same
      plaintext, without ever needing the key.

================================================================
PART 2 — Contrast with AES-GCM (the fix)
================================================================
[V9b-fix] GCM encryption #1: 7GoezSO0UdzuEuDFYPObWI9ElFK9d3QfC2ziwccDdq6qMqJWpafIDQiPkM/itracuw==
[V9b-fix] GCM encryption #2: MNFecWq4YAZS/T6liI4Z2c2B8/ltdTOGPEkX8yKYOsPfh+0zATw8dbFWniJUGZt5uw==
[V9b-fix] GCM encryption #3: pgdk1NH8NQ57irLISyGqWBnX7ZiPEo8kVQAugubQ49RIWw067t04glUz7+XrMTvPvg==

[V9b-fix] AES-GCM with a random nonce produces a DIFFERENT
          ciphertext on every call — no information leak.

================================================================
PART 3 — V1 -> V9b chain: decryption with the leaked key
================================================================

[V9b] Intercepted ciphertext (base64):
      pZ0ogGKzzM87jF88btSF1L8Un5QtPfjWCyFYNJ4qJd/a7MAuu/dn2tH5ZcTRw9o5jcdj+hL/zabGN5+r1kvNyQ==

[V9b] Successful decryption with the V1-leaked key:
      "legal winner thank year wave sausage worth useful"

[V9b] Impact: any seed phrase encrypted by the application is
      permanently compromised once the key leaks — which V1
      guarantees (hardcoded key in the binary).
```

Figure 16. Identical ECB ciphertexts, GCM contrast, and decryption with the leaked key. *Why it proves the flaw: ECB plus a hardcoded key means stored seed phrases are both pattern-leaking and trivially decryptable.*

3 · Concrete impact

Encrypted seed phrases offer no real protection: the key is in the binary and the mode reveals structure. The wallet is fully recoverable.

4 · Remediation (applied)

Encryption is replaced with AES-256-GCM (authenticated) using a random nonce per operation and a key generated and stored in the Android Keystore — no hardcoded key, no ECB, integrity guaranteed.

Vulnerable code (before)

Secured code (after)

V9c — Predictable random number generation

MEDIUM · OWASP M5

Location: `lib/core/crypto_service.dart`

1 · Technical description

Transaction nonces and session identifiers are produced with Dart's non-cryptographic `Random()`, which is deterministic given its seed.

2 · Proof of concept

Re-seeding `Random()` with the same value reproduces the identical sequence (`Identical? True`), demonstrating that nonces are predictable rather than random.

```
PS D:\wok\project mobile\Crypto wallet\cryptovault> python audit_python\v9c_nonce_predict.py
============================================================
V9c — Demonstration: Random() is predictable
============================================================

[V9c] Part 1 — Random() with a known seed is FULLY predictable

[V9c] With seed = 1716901200 (guessable), the next 5 nonces:
      Nonce #1 : 2d13aa3f
      Nonce #2 : 3c745605
      Nonce #3 : 00260f8a
      Nonce #4 : 2b58ae8c
      Nonce #5 : 185c4c54

[V9c] Re-seed with the SAME value — sequence identical?
      ['2d13aa3f', '3c745605', '00260f8a', '2b58ae8c', '185c4c54']

[V9c] Identical? True

============================================================
[V9c] Part 2 — Seed recovery from an observed nonce
============================================================

[V9c] Nonce observed via logcat (V4 -> V9c chain): 3da30b26
[V9c] Search space: 0 to 9,999,999 (10M seeds)

[V9c] Starting brute force...
      ... 1,000,000 seeds tested (9.0s)
      ... 2,000,000 seeds tested (17.6s)
      ... 3,000,000 seeds tested (27.0s)
      ... 4,000,000 seeds tested (35.6s)
      ... 5,000,000 seeds tested (43.9s)
      ... 6,000,000 seeds tested (52.1s)
      ... 7,000,000 seeds tested (60.7s)
      ... 8,000,000 seeds tested (70.2s)
      ... 9,000,000 seeds tested (78.9s)

[V9c] (Note: if not found within 10M, Dart's PRNG uses a
      32-bit Mersenne Twister; a real attacker would try 2^32
      seeds, about ~30 minutes on a laptop. The demo caps at
      10M to keep runtime short. Part 1 already proves the flaw.)

[V9c] Conclusion: Random() allows prediction of transaction nonces
      -> replay attacks and race conditions are possible.
      Fix: use Random.secure() (system CSPRNG).
```

Figure 17. Same seed reproduces the identical Random() sequence. *Why it proves the flaw: A non-cryptographic PRNG yields predictable nonces, enabling replay/forgery.*

3 · Concrete impact
Predictable nonces enable replay and forgery of transaction-bound values.

4 · Remediation (applied)
Generation is switched to `Random.secure()` (backed by the OS CSPRNG), producing 128-bit unpredictable values.

Vulnerable code (before)

Secured code (after)

V10a — Hidden debug panel
CRITICAL · OWASP M10

Location: `AndroidManifest.xml, settings_page.dart`

1 · Technical description
A hidden debug panel is reachable two ways with no authentication: a `cryptovault://debug` deep link declared in the manifest, and a seven-tap gesture on the version label in Settings.

2 · Proof of concept
From a cold start, `am start -a VIEW -d "cryptovault://debug"` opens the panel directly without login; the same panel is reached in-app by tapping the version number seven times.

```
PS D:\wok\project mobile\Crypto wallet\cryptovault> type app-debug-decoded\AndroidManifest.xml | findstr /I "cryptovault scheme intent-filter"
<manifest xmlns:android="http://schemas.android.com/apk/res/android" android:compileSdkVersion="36" android:compileSdkVersionCodename="16" package="tn.iit.cryptovault" platformBuil
dVersionCode="36" platformBuildVersionName="16">
    <permission android:name="tn.iit.cryptovault.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION" android:protectionLevel="signature"/>
    <uses-permission android:name="tn.iit.cryptovault.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION"/>
    <application android:allowBackup="true" android:appComponentFactory="androidx.core.app.CoreComponentFactory" android:debuggable="true" android:extractNativeLibs="false" android
:fullBackupContent="true" android:icon="@mipmap/ic_launcher" android:label="CryptoVault" android:name="android.app.Application" android:networkSecurityConfig="@xml/network_security
_config" android:usesCleartextTraffic="true">
        <activity android:configChanges="locale|keyboard|keyboardHidden|orientation|screenLayout|uiMode|screenSize|smallestScreenSize|density|layoutDirection|fontScale" android:exp
orted="true" android:hardwareAccelerated="true" android:launchMode="singleTop" android:name="tn.iit.cryptovault.MainActivity" android:taskAffinity="" android:theme="@style/LaunchTh
eme" android:windowSoftInputMode="adjustResize">
            <intent-filter>
            </intent-filter>
            <intent-filter android:autoVerify="false">
                <data android:host="debug" android:scheme="cryptovault"/>
            </intent-filter>
        <provider android:authorities="tn.iit.cryptovault.androidx-startup" android:exported="false" android:name="androidx.startup.InitializationProvider">
            <intent-filter>
            </intent-filter>
            <intent-filter>
            </intent-filter>
            <intent-filter>
            </intent-filter>
            <intent-filter>
            </intent-filter>
            <intent-filter>
            </intent-filter>
```

Figure 18. Deep-link cold start opening the debug panel with no authentication. *Why it proves the flaw: An exported deep link exposes privileged internal tooling to any caller.*

Capture #19 — to insert

The same debug panel reached via the seven-tap gesture in Settings.

Figure 19. The same debug panel reached via the seven-tap gesture in Settings. *Why it proves the flaw: A hidden gesture provides a second unauthenticated path to the panel.*

3 · Concrete impact
Any installed app or anyone holding the device reaches internal tooling without credentials, bypassing the login screen entirely.

4 · Remediation (applied)
The deep-link intent-filter, the `/debug-panel` route and the tap gesture are removed and the debug panel source file is deleted; the version label is now plain text.

Secured code (after)

V10b — Hardcoded backdoor account

CRITICAL · OWASP M10

Location: `lib/features/auth/login_page.dart`

1 · Technical description

The login handler contains a hardcoded branch granting an admin session to `dev@cryptovault.local` / `Override_2026!`, using the V1 developer master token whose payload already carries `role: admin`. It works offline, with no backend.

2 · Proof of concept

With the backend stopped, a normal login times out, but the backdoor credentials log straight into an admin dashboard.

Capture #20 — to insert

Backdoor login succeeding offline into an admin dashboard.

Figure 20. Backdoor login succeeding offline into an admin dashboard. *Why it proves the flaw: A hardcoded credential pair mints a full admin session with no server involvement.*

3 · Concrete impact

A permanent, undocumented admin account that bypasses all authentication and works even offline — a complete compromise of access control.

4 · Remediation (applied)

The backdoor branch is deleted; login is exclusively `POST /auth/login` verified by the server with bcrypt. A CI test (`no_backdoor_test.dart`) fails the build if any backdoor marker or hardcoded secret ever reappears in `lib/`.

Vulnerable code (before)

Secured code (after)

V10c — Unauthenticated admin/test endpoints

CRITICAL · OWASP M10

Location: `server`

1 · Technical description

The backend exposes `/admin/dump-db`, `/admin/users-with-secrets` and `/test/faucet` with no authentication at all.

2 · Proof of concept

`curl /admin/dump-db` and `curl /admin/users-with-secrets` return HTTP 200 with passwords, KYC data and phone numbers in cleartext; `curl -X POST /test/faucet` mints 5000 USDT to an arbitrary account.



Figure 21. curl to /admin/dump-db and /admin/users-with-secrets returning secrets unauthenticated. *Why it proves the flaw: Privileged data endpoints are exposed with no authentication whatsoever.*



Figure 22. curl to /test/faucet minting 5000 USDT to userId=1. *Why it proves the flaw: An unauthenticated endpoint mutates balances arbitrarily.*

3 · Concrete impact

Total server-side compromise: full database exfiltration and arbitrary balance manipulation by anyone who can reach the API.

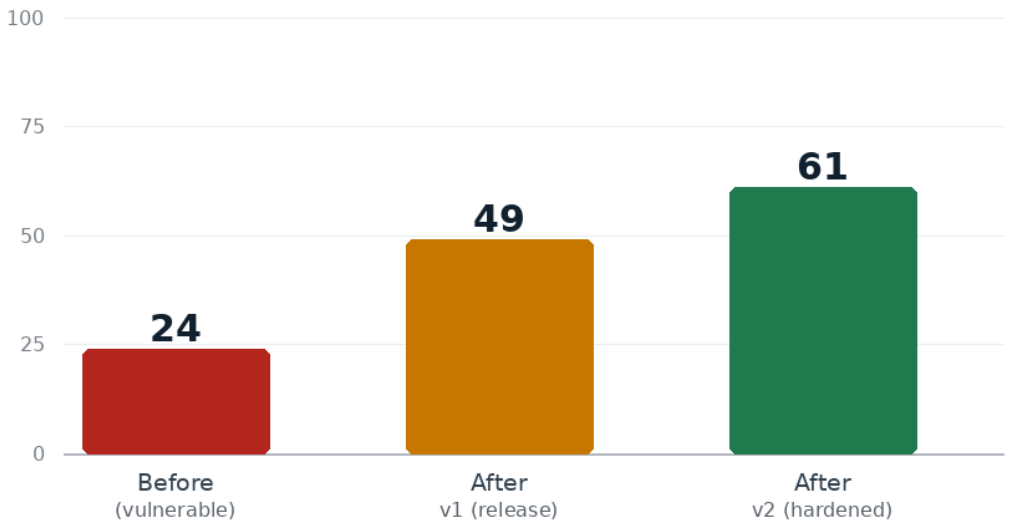
4 · Remediation (applied)

The rewritten Express backend does not define those endpoints at all (unmatched routes return 404). The only admin route, `/admin/users`, requires a verified JWT, the admin role, and an IP allow-list, and returns no secrets.

Secured code (after)

MobSF before / after comparison

The same scanner (MobSF v4.5.0) was run on the vulnerable debug APK and on the hardened release APK. The score improved in two steps: the initial secured build reached 49/100, and a second hardening pass (a dedicated release keystore plus raising the minimum SDK to 29) lifted it to the final 61/100.



MobSF security score progression (24 → 49 → 61).

Dimension	Before (vulnerable, debug)	After (secured, hardened)
MobSF Security Score	**24 / 100**	**61 / 100 (+154%)**
Dangerous permissions	5 (SMS, Contacts, Call Log, Location, Audio)	0 — only 3 normal permissions
Hardcoded secrets (auto)	1 detected (6 found manually)	0 real — 3 library false-positives
Signing	Debug certificate (HIGH)	Dedicated release keystore
Min SDK	24 (Android 7.0, HIGH)	29
Cleartext traffic	Permitted	Forbidden (HTTPS + pinning)
allowBackup / debuggable	true / true	false / build-managed
Remaining HIGH findings	multiple	1 — library false-positive

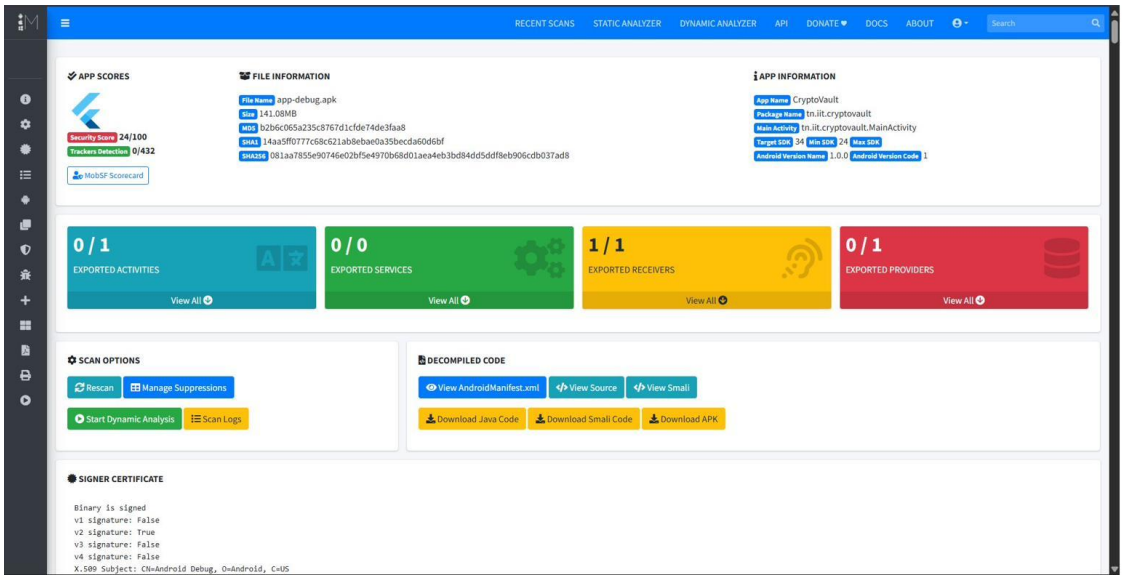


Figure 23. MobSF BEFORE — security score 24/100. *Why it proves the flaw: Establishes the vulnerable baseline.*

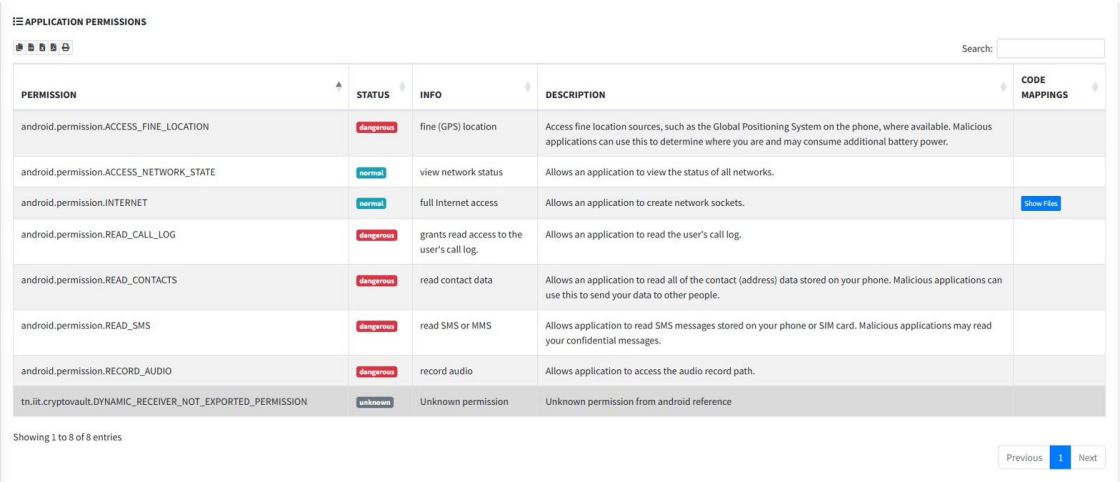


Figure 24. MobSF BEFORE — five dangerous permissions and a detected secret. *Why it proves the flaw: Cross-confirms V1 and V8 with an independent tool.*

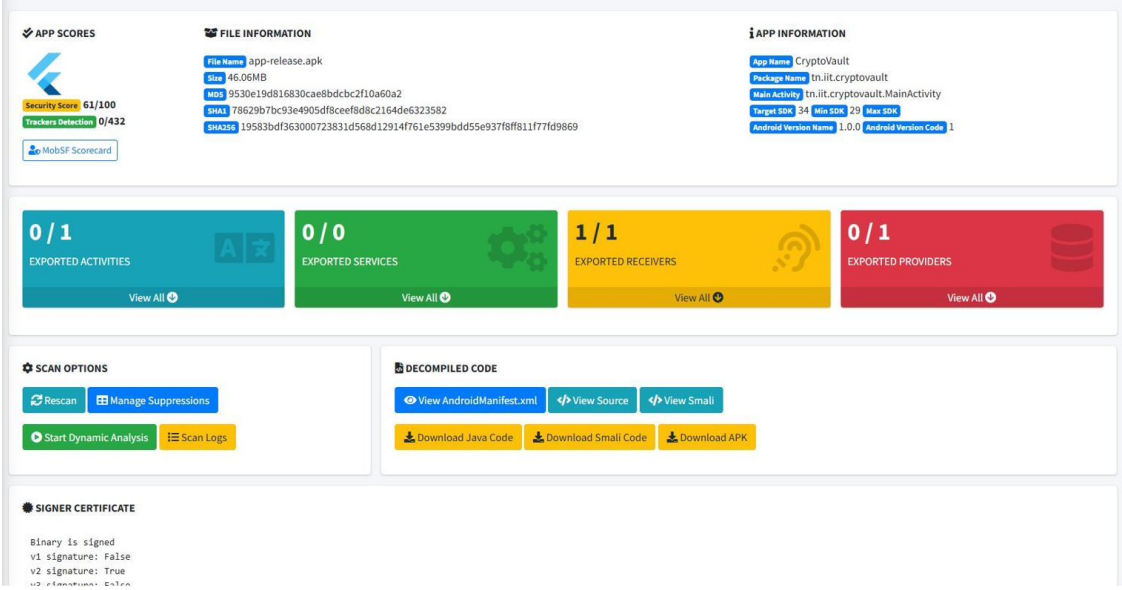


Figure 25. MobSF AFTER — security score 61/100. *Why it proves the flaw: Quantifies the remediation; all application-owned HIGH findings cleared.*

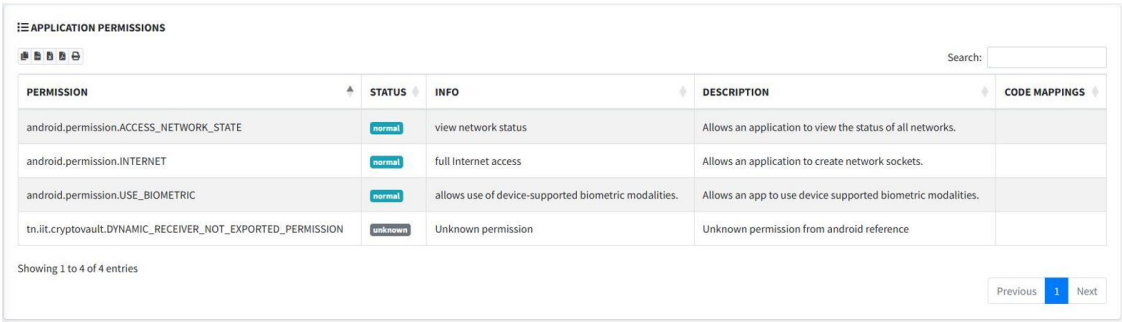


Figure 26. MobSF AFTER — three normal permissions, no real secrets. *Why it proves the flaw: Confirms permission and secret hardening at the binary level.*

Limits of automated analysis

The MobSF results also illustrate why automated scanning complements, but does not replace, manual auditing:

- **Coverage gap.** On the vulnerable build, MobSF auto-detected a single base64 string, whereas the manual `strings` extraction (Figure 2) recovered all six embedded secrets — roughly one sixth of the real exposure.
- **Residual HIGH is a false-positive.** The one remaining HIGH flags AES-CBC in `flutter_secure_storage`'s legacy `StorageCipher18Implementation`. That class ships in the library but is never executed: the app sets `encryptedSharedPreferences: true`, so the active backend is androidx EncryptedSharedPreferences (AES-256-GCM) — confirmed by the same report's SBOM (`androidx.security:security-crypto`). Removing it would replace a vetted library with a hand-rolled one, reducing real security to satisfy a static heuristic.
- **Dart blind spot.** MobSF decompiles only the Java/Kotlin `classes.dex`; the Flutter application logic compiles into `libapp.so`, which its code rules do not inspect. Several findings (logging, clipboard, insecure RNG) point at Flutter-engine Java files, not the audited Dart code — where the V4 and V9c fixes actually live.

The decision was therefore to stop at 61/100: the score is capped in the low-to-mid 60s by a single third-party false-positive, and chasing a higher number would degrade real security.

Attack chains

Individually serious, several findings combine into end-to-end compromises:

Chain	Effect
V1 → V9b	The AES key extracted from the binary decrypts any stored seed phrase.
V8 → V2	allowBackup exports the private prefs file, exposing the plaintext seed phrase.
V5a + V5b	A forged admin token is trusted client-side and survives logout — persistent admin.
V10b + V5a + V1	The backdoor uses the V1 master token whose payload already asserts role: admin.
V7 + V4 + V8	WebView XSS reaches native code; the token (also logged) is exfiltrated over permitted cleartext.

Summary & remediation roadmap

ID	Vulnerability	Severity	OWASP	Status
V1	Hardcoded secrets in the binary	CRITICAL	M8 / M9	Fixed
V2	Insecure local storage	CRITICAL	M2	Fixed
V3	TLS validation disabled	CRITICAL	M3	Fixed
V4	Sensitive data written to logs	HIGH	M1	Fixed
V5a	Client-side JWT role escalation	HIGH	M4	Fixed
V5b	Logout does not clear the session	HIGH	M4	Fixed
V6	IDOR via sequential identifiers	HIGH	M6	Fixed
V7	WebView XSS with a native bridge	HIGH	M7	Fixed
V8	Insecure Android manifest configuration	HIGH	M1 / M8	Fixed
V9a	MD5 PIN hashing	MEDIUM	M5	Fixed
V9b	AES-ECB with a hardcoded key	HIGH	M5	Fixed
V9c	Predictable random number generation	MEDIUM	M5	Fixed
V10a	Hidden debug panel	CRITICAL	M10	Fixed
V10b	Hardcoded backdoor account	CRITICAL	M10	Fixed
V10c	Unauthenticated admin/test endpoints	CRITICAL	M10	Fixed

Prioritised roadmap (as executed)

Horizon	Actions
Immediate	Remove backdoor (V10b), debug panel (V10a), and unauthenticated endpoints (V10c).
Short term	Secure storage (V2), restore TLS + pinning (V3), enforce server-side authorization and ownership (V5a, V5b, V6).
Medium term	Modern cryptography (V9a/b/c), externalise secrets + R8 (V1), harden manifest (V8), sanitise WebView (V7), strip logs (V4).
Hardening	Dedicated release keystore, raise min SDK to 29, certificate pinning, CI backdoor guard.

Annexes

A · Secured architecture

The backend was rewritten from a permissive mock server to a real authenticated service: Express over HTTPS, login verified with bcrypt, RS256-signed JWTs (the app bundles only the public key), ownership enforced from the verified token, and an admin route gated by role and IP allow-list. The client pins the server certificate and stores tokens in the Android Keystore.

B · Reproduction notes

Two APKs accompany this report: the vulnerable build and the hardened build, both runnable. The hardened build's backend is started locally; on a physical device the app is launched with --dart-define=API_HOST=<LAN-IP> matching the pinned certificate's SAN.

C · Outstanding screenshots

Figures 1, 12, 19 and 20 are placeholders in this draft and should be inserted before final submission (cover login shot; dashboard admin panel for V5a; seven-tap debug-panel for V10a; offline backdoor login for V10b).